Measuring and Improving Native Execution in Apache DataFusion Comet

Milestone 2 Design Doc

Pranav Chintareddy, Selena Liu, Adam Yanai, Yongyi Xie, Joseph Attie

Demo Setup + Execution

To replicate demo results:
Requirements
- Docker Desktop (Mac/Windows) or Docker Engine (Linux)
- At 10 GB disk space recommended
- Provided Docker image

Quick start
```bash
# Step 1: build the image (may take 30+ min)
docker compose build

# Step 2: run the benchmark
docker compose run comet-demo bash /opt/scripts/run_demo.sh
```

What the build does
1. Installs Java 17, Rust, Maven
2. Downloads Spark 3.5.8
3. Builds Comet 0.15.0-SNAPSHOT from source with implemented TPC-H fixes
4. Compiles `tpch-dbgen` and generates TPC-H SF=1 data
5. Converts `.tbl` files to Parquet

Background and Motivation

Running all 22 TPC-H queries against Spark 3.5.8 with Comet established a baseline. The vast majority of queries run fully natively in under 2 seconds each, with two partially falling back to JVM (Q16 and Q22). One query stood out as a severe outlier: Q21 took approximately 33 seconds, more than 10x slower than expected. Investigation of the Comet fallback logs revealed two separate root causes both affecting Q21.

Implementations

TPC-H Fixes - Q21 (SortMergeJoin Inequality Fast Path), Q16, Q22

Problem

TPC-H Q21 contains two correlated subqueries using EXISTS and NOT EXISTS, which Spark compiles into a chain of LeftSemi and LeftAnti SortMergeJoins with an inequality filter on the supplier key column (l_suppkey <> l1.l_suppkey).

In the original execution path, for every row on the streamed side the engine scans every row on the buffered side to evaluate the filter. On the 6-million-row lineitem table this is an O(S × R) operation and dominates total query time, producing the observed ~33 second runtime.

Design

The fix introduces a fast path for LeftSemi and LeftAnti SortMergeJoins that have a single binary inequality filter. At join construction time, the code inspects the filter expression. If it is a simple binary inequality (<, <=, >, >=, <>) between one column from each side, a SemiAntiFastPathSpec is created recording the operator kind and the filter column names.

At runtime, before entering the O(S × R) scan loop, the code checks whether a fast path spec exists. If so, it scans the buffered group once to find a single representative value — the minimum value for < and <= operators, the maximum value for > and >=, or the first non-null value for <>. It then evaluates the join filter against just that one (streamed row, representative buffered row) pair. Based on the result, the streamed row is either emitted directly or skipped, without touching any remaining buffered rows.

This converts the per-streamed-row work from O(buffered group size) to O(1), giving overall O(S + R) complexity per join group.

The fast path is gated behind a FilteredSmjMode configuration flag so it can be toggled at Spark startup without recompiling. The implementation lives in datafusion/physical-plan/src/joins/sort_merge_join/stream.rs and exec.rs in the DataFusion library.

Result

Q21 runtime dropped from approximately 33 seconds to approximately 2.5 seconds, an over 10× speedup with no change to query correctness. Also, Q16, Q22 queries used to previously fall back to Spark JVM as there was no native Comet support for the ‘Build Right with LeftAnti’ case and there used to be warnings for Q16 & Q22. But now, there are no warnings for any of the queries. Their latency(Q16,Q22) has been reduced by HALF. For sake of context, this was how it used to look PREVIOUSLY before our code changes(notice the Q16,Q21,Q22 where you can see the higher latency compared to Video’s latency as well as warnings in the NEXT page below):

```
FINAL OUTPUT (Time in s will differ with consecutive runs):
Q1      0.400 s
Q2      0.495 s
Q3      0.639 s
Q4      0.405 s
Q5      1.017 s
Q6      0.161 s
Q7      1.745 s
Q8      0.830 s
Q9      1.259 s
Q10     0.650 s
Q11     0.460 s
Q12     0.478 s
Q13     0.791 s
Q14     0.305 s
Q15     0.488 s
26/02/16 22:00:34 WARN CometExecRule: Comet cannot execute some parts of this plan natively (set spark.comet.explainFallback.enabled=false to disable this logging):
CometSort
+- CometSinkPlaceHolder
   +- CometExchange
      +- CometHashAggregate
         +- CometSinkPlaceHolder
            +- CometExchange
               +- CometHashAggregate
                  +- CometHashAggregate
                     +- CometSinkPlaceHolder
                        +- CometColumnarExchange
                           +- HashAggregate
                              +- Project
                                 +- BroadcastHashJoin
                                    :-  BroadcastHashJoin [COMET: BuildRight with LeftAnti is not supported]
                                    :  :- CometFilter
                                    :  :  +- CometScanWrapper
                                    :  +- BroadcastExchange
                                    :     +- CometProject
                                    :        +- CometFilter
                                    :           +- CometScanWrapper
                                    +- BroadcastExchange
                                       +- CometFilter
                                          +- CometScanWrapper

Comet accelerated 17 out of 23 eligible operators (73%). Final plan contains 0 transitions between Spark and Comet.
Q16     0.804 s
Q17     0.567 s
Q18     0.909 s
Q19     0.338 s
Q20     0.427 s
Q21    35.444 s
26/02/16 22:01:13 WARN CometExecRule: Comet cannot execute some parts of this plan natively (set spark.comet.explainFallback.enabled=false to disable this logging):
CometSort
+- CometSinkPlaceHolder
   +- CometColumnarExchange
      +- HashAggregate
         +- CometSinkPlaceHolder
            +- CometColumnarExchange
               +- HashAggregate
                  +- Project
                     +-  BroadcastHashJoin [COMET: BuildRight with LeftAnti is not supported]
                        :- CometFilter
                        :  :  +- Subquery
                        :  :     +- CometColumnarToRow
                        :  :        +- CometHashAggregate
                        :  :           +- CometExchange
                        :  :              +- CometHashAggregate
                        :  :                 +- CometProject
                        :  :                    +- CometFilter
                        :  :                       +- CometScan [native_iceberg_compat] parquet spark_catalog.default.customer_parquet
                        :  +- CometScanWrapper
                        +- BroadcastExchange
                           +- CometScanWrapper

Comet accelerated 14 out of 20 eligible operators (70%). Final plan contains 1 transitions between Spark and Comet.
Q22     1.304 s
```